

Technics SX-WSA1R

System Exclusive Reference

Reconstructed from the OS v2.0 firmware

Acoustic Modeling Synthesizer Module, 1995

Every claim in this document carries a certainty marker and the firmware symbol that establishes it.

Technics published no MIDI implementation chart for this instrument.

Contents

1	Scope and provenance	3
1.1	What this document is	3
1.2	Which firmware	3
1.3	How to read the certainty markers	3
1.4	One correction to earlier project notes	4
2	Message framing	5
2.1	The identifier test	5
2.2	How a message is tracked	5
2.3	What happens to another maker's message	6
2.4	Termination, abort, and real time	6
3	The Matsushita message set	7
3.1	The third byte	7
3.2	The model triple	7
3.3	Short forms	7
3.4	The parameter families	8
4	Bulk dump	9
4.1	What can be dumped	9
4.2	Frame layout	9
4.3	Nibble encoding	10
4.4	Checksum	10
4.5	Handshake	10
4.6	Per-frame acknowledgement and timeouts	10
4.7	Session control messages	11
4.8	Continuation frames	11
4.9	Receiving	11
5	Universal System Exclusive	12
5.1	What is recognised	12
5.2	What is not recognised	12
5.3	Two caveats	12
6	Error reporting	13
6.1	The chain	13
6.2	ERROR 40 — wrong identification	13
6.3	ERROR 41 — reception	13
6.4	ERROR 42 — transmission	14
6.5	Where errors are visible	14

6.6	Verbatim screen text	14
A	Evidence index	15
A.1	Technics SX-WSA1R ? Universal / General MIDI System Exclusive (receive and transmit), from the annotated TLCS-	15
A.2	SX-WSA1R MIDI transport on CPU 1 (prom_a/prom_b): how MIDI 1 and MIDI 2 are distinguished, what confines BULK	17
A.3	Technics SX-WSA1R firmware ? MIDI System Exclusive RECEIVE path (prom_a interrupt parser, prom_a foreground pa	19
A.4	WSA1R SysEx error reporting ? where ERROR 40 / 41 / 42 are raised (prom_a + prom_b)	20
A.5	Technics SX-WSA1R firmware ? the SysEx BULK DUMP transmit path (prom_a UI screen 0x79 → the SysEx engine at 0x	22
B	Open questions	25

Chapter 1

Scope and provenance

1.1 What this document is

This is a byte-level reference for the System Exclusive implementation of the Technics SX-WSA1R, reconstructed by reading the instrument's own firmware. Technics published a user guide describing the SYSEX BULK DUMP feature and its error numbers, but no MIDI implementation chart and no message format. Everything in the following chapters was recovered from the program ROMs.

1.2 Which firmware

All addresses and all behaviour described here are those of the **OS v2.0** images:

image	chip	factory part	role
prom_a	IC12	QSIGCWSA1AX	CPU 1, MIDI and panel
prom_b	IC13	QSIGCWSA1BX	CPU 1, tables and display lists
prom_c	IC28	QSIGCWSA1CX	CPU 2
prom_d	IC21	QSIGCWSA1DX	tone bank

A v1 OS shipped and is not dumped. Nothing here should be assumed to hold for it.

1.3 How to read the certainty markers

Every substantive claim carries one of three markers.

established Read directly from instructions or tables in the firmware. The symbol and address that establish it are cited.

likely Follows from the code but was not executed or exhaustively checked.

unverified Stated in the firmware's own annotations or in Technics' documentation, but not confirmed here.

A wrong byte value in a document like this is worse than a gap, so where the evidence stops, the text says so rather than rounding up to a plausible answer.

1.4 One correction to earlier project notes

The label `MIDI_SysExHeader` does not do what its name suggests

`MIDI_SysExHeader` at `0xFA5CB8` contains the two bytes `F0 50`, and it is tempting — this project did it — to cite it as the manufacturer-identifier test. It is not. It is an *outgoing* template, and no compare instruction anywhere in either CPU 1 image reads that address. The acceptance decision is made by two immediate comparisons inside `MIDI_RX_SysExStart`; see section 2.1. The resulting value, `0x50`, happens to be the same. The evidence is not.

Chapter 2

Message framing

2.1 The identifier test

established The instrument accepts exactly two System Exclusive identifiers, and they are literal immediate operands, not settings:

identifier	meaning	instruction
0x50	Matsushita (Technics)	<code>cp E,0x50 at 0xFA5850</code>
0x7E	Universal Non-Real Time	<code>cp E,0x7E at 0xFA5855</code>

There is no third comparison, no table lookup, and no read of any stored setting on this path. **There is no device or unit number at this level** — the byte after the identifier is already treated as message body.

2.2 How a message is tracked

established 0xF0 is first handled as an ordinary status byte and stored as running status. The SysEx logic runs when the *next* byte arrives, so the byte the identifier test inspects is message byte index 1.

A single RAM byte, (0xA9), holds the whole receive state:

bit	meaning	set at
0	armed — an F0 was seen	0xFA584D
1	identifier accepted	0xFA585A
2	a message completed cleanly (byte value 0x04)	0xFA54E2
4	aborted by a stray status byte (byte value 0x10)	0xFA54EE
5	suppress further SysEx	0xFB6151

established A UART receive error clears the byte outright at 0xFA5426.

A correction to the firmware's own annotations

Both the block header above `MIDI_RX_SysExStart` and the project's `FINDINGS-midi-port.md` state that bit 5 is “set nowhere in the converted code”. It *is* set, at 0xFB6151, in the synchronous receiver on a SysEx framing error. Bits 2 and 4 are written but no reader of them was found anywhere in `prom_a`.

2.3 What happens to another maker's message

established An identifier that is neither 0x50 nor 0x7E leaves bit 0 set and bit 1 clear. Every following byte still reaches the data handler and is discarded there, so the instrument tracks the foreign message to its F7 without acting on it and without raising any error. Outside a bulk-dump session this is silent.

2.4 Termination, abort, and real time

established Termination is decided in the status-byte path, not in the SysEx routines:

- F7 with bit 5 clear forwards the F7 and leaves (0xA9) = 0x04, the “complete message received” residue.
- Any other status byte in 0x80–0xF6 aborts: (0xA9) = 0x10 and nothing is forwarded.
- System Real Time (0xF8–0xFF) is split off earlier, at 0xFA54B6, so a clock or active-sensing byte *inside* a System Exclusive message does not break it. This is correct MIDI 1.0 behaviour.

established Conversely, while a SysEx is armed or in progress the instrument ignores 0xF8 TIMING CLOCK, 0xFA START, 0xFB CONTINUE and 0xFC STOP for transport purposes (0xFA5520). The block sender likewise spins until an inbound message finishes before transmitting (0xFB99CC).

Chapter 3

The Matsushita message set

3.1 The third byte

established After F0 50, the third byte selects a message family. It is matched against a 15-record table at **prom_b** 0xF5115B, of which 14 are live and the fifteenth is an 0xFF terminator. The accepted values are

21 22 23 24 25 27 28 29 2A 2B 2C 2D 7E 7F

established 0x26 is *not* accepted. The WSA1's own product identifier is 0x2D: every outgoing bulk-dump header in **prom_b** begins F0 50 2D 04 00 11.

3.2 The model triple

established The families 0x21, 0x22, 0x2B, 0x2C and 0x2D all continue with three bytes: 0x04, then *either* 0x00 *or* 0x01, then 0x11. Both middle values are separate literal records in the grammar — they are not read from a setting, a strap or NVRAM at match time.

established The enquiry handler copies the responding unit's three bytes into RAM 0x60FC90–0x60FC92. No reader of those cells was found anywhere in **prom_a**, so the remote unit's identity is remembered and, in this firmware, never used.

3.3 Short forms

established These sequences terminate within a few bytes. Each is also present verbatim as an outgoing template in **prom_b** between 0xF4FEB4 and 0xF4FF60, which corroborates the grammar independently.

bytes	meaning	internal command
F0 50 23 7E F7	acknowledge (ACK)	0x01
F0 50 24 7E F7	not acknowledged (NAK)	0x02
F0 50 27 7E F7	end of category	0x03
F0 50 28 7E F7	end of dump	0x04
F0 50 29 7E F7	abort / gave up	0x05
F0 50 2A 7E F7	status 0x16	0x04
F0 50 21 04 <i>nn</i> 11 F7	enquiry	0x07
F0 50 22 04 <i>nn</i> 11 F7	enquiry reply	0x08
F0 7E 7F 09 01 F7	GM System On	0x20
F0 7E 7F 09 02 F7	GM System Off	0x21

where *nn* is 0x00 or 0x01.

established 0x2A yields the same internal command number as 0x28.

3.4 The parameter families

established 0x2B and 0x2C each open into a large parameter-address subtree — 3756 accepted byte sequences apiece — and 0x2D carries 16, ending in internal command ids 0x0B, 0x0C, 0x0E and 0x12–0x16. A matched sequence yields a command number bounded by `cpA,0x22` and used to index one of two 34-entry handler tables at 0xF4F800 and 0xF4F888.

likely The 0x2B and 0x2C families are single-parameter traffic rather than bulk transfer. Their individual parameter addresses are not enumerated in this edition.

Chapter 4

Bulk dump

Technics' user guide states that SYSEX BULK DUMP exchanges the instrument's internal data as System Exclusive, that the instrument is silent during the transfer, and that it works on the **MIDI 1** terminals only. This chapter gives the wire protocol.

4.1 What can be dumped

established The SYSEX BULK DUMP screen offers five categories. Each selects a job code which selects a dump routine.

row	category	job	routine
0	TOTAL KEYBOARD	0	0xFB22CD
1	SOUND	3	0xFB22F6
2	COMBINATION	5	0xFB2314
3	SYSTEM, PART & MIDI	4	0xFB2305
4	SEQUENCER	2	0xFB22E7

Reproduce with `notes/sysex-probes/sysex_bulkdump_tx.py`.

4.2 Frame layout

established A data frame is

`F0 50 2D 04 nn 11 a2 a1 a0 l2 l1 l0 [data...] c sum F7`

nn 0x00 or 0x01, the unit field.

a2 a1 a0 destination address, three bytes, base 128 — the value is $a_2 \cdot 128^2 + a_1 \cdot 128 + a_0$.
established

l2 l1 l0 length in *source* bytes, same base-128 encoding.

data the payload, nibble encoded (below).

c continuation flag: 0x01 if more frames follow, 0x00 on the last.

sum checksum (below).

established A frame is filled until its byte count reaches `0xFC = 252`.

4.3 Nibble encoding

established Every source byte becomes two message bytes, **high nibble first**:

$$b \longrightarrow (b \gg 4), (b \& 0x0F)$$

So source byte 0xF1 is sent as 0F 01, and a dump occupies twice as many bytes on the wire as the length field states. On receive the two are rejoined as $((\text{first} \& 0x0F) \ll 4) | (\text{second} \& 0x0F)$. An odd body length is a hard error.

4.4 Checksum

established Let S be the 8-bit sum of every byte from the 0x50 identifier through the continuation flag inclusive. Then

$$\text{sum} = (0 - S) \& 0x7F$$

The leading F0, the checksum byte itself and the trailing F7 are outside the sum. Equivalently, the sum of everything after F0 including the checksum is 0 mod 128.

The checksum is not always verified

established On receive the checksum is only checked when the matched product id is one of 0x7E, 0x2B, 0x2C or 0x2D. For any other accepted family *no checksum check is performed at all*.

4.5 Handshake

established Every dump job begins with a two-step enquiry, each step retried up to three times.

1. Send F0 50 21 04 00 11 F7; expect F0 50 22 04 *nn* 11 F7. On success the reply's three identity bytes are stored.
2. Send F0 50 22 04 00 11 F7; expect F0 50 23 7E F7 (ACK). On success per-frame acknowledgement is enabled for the rest of the dump.

established If all three attempts of step 1 fail, the error is cleared and **the dump proceeds without the acknowledgement protocol**.

4.6 Per-frame acknowledgement and timeouts

established With the handshake established, each data frame must be answered with F0 50 23 7E F7. Anything else causes the same frame to be retransmitted, up to three attempts, after which the session aborts.

established Without the handshake no acknowledgement is expected: the sender waits for the MIDI output ring to drain and then pauses 25 timer ticks — about 51ms at the instrument's 488.28 Hz tick — before the next frame.

established A reply wait times out after 2500 ticks (about 5.1s), or 1000 ticks (about 2.05s) during the handshake.

4.7 Session control messages

established After each category the sender emits F0 50 27 7E F7 and waits for the ACK. After the whole job it emits F0 50 28 7E F7. If three retries have failed it then emits F0 50 29 7E F7, the abort notification.

4.8 Continuation frames

established Frames after the first re-open with F0 50 7E, followed by data, the continuation flag, the checksum and F7.

4.9 Receiving

established The same protocol runs in reverse. The instrument answers a data frame with F0 50 23 7E F7 for ACK, F0 50 24 7E F7 for NAK, and F0 50 2A 7E F7 to report status 0x16.

established Inside a session the byte-level framer is stricter than the general receiver: it accepts *only* 0x50 as the identifier, not 0x7E.

Chapter 5

Universal System Exclusive

5.1 What is recognised

established The instrument recognises exactly **two** universal messages:

bytes	meaning
F0 7E 7F 09 01 F7	GM System On
F0 7E 7F 09 02 F7	GM System Off

established Both are also *transmitted* by the instrument, from literals at 0xF4FEE6 and 0xF4FEEC, and both are written into Standard MIDI Files as a delta-time-zero five-byte SysEx event (00 F0 05 7E 7F 09 01 F7).

established GM System Off is a no-op when GM mode is already off.

5.2 What is not recognised

established The following are rejected by the grammar. A device expecting any of them will get no response:

- F0 7F 7F 04 01 *ll mm* F7 — Master Volume. The whole 0x7F Universal *Real Time* identifier is unmatched at byte 1.
- F0 7E *dd* 06 01 F7 — Identity Request. There is no identity reply in this firmware.
- F0 7E 7F 09 03 F7 — GM2 System On.

5.3 Two caveats

likely Trailing bytes before the F7 are probably tolerated, because the grammar stops matching once a command is resolved.

likely The identifier byte is not re-checked after the front-end accepted it, so F0 50 7F 09 01 F7 would probably act as GM System On despite carrying the Matsushita identifier. Neither caveat was executed on hardware or in emulation.

Chapter 6

Error reporting

6.1 The chain

established None of the three documented errors is raised as the number 40, 41 or 42. The SysEx engine keeps a parse record whose field 4 is a status byte. At the end of a bulk-dump session that status is mapped through a table to a message id, which selects a display list.

message id	screen	raised by status
0x23	COMPLETED!	0x00
0x22	ERROR 40!	0x08, 0x09, 0x0A
0x21	ERROR 41!	everything not listed elsewhere
0x20	ERROR 42!	0x20 (and 0x18, 0x1F, never raised)
0x0F	ERROR 21! “Memory full”	0x16

Not every bulk-dump failure is a 4x error

established Status 0x16 — the incoming dump is larger than the destination area — produces ERROR 21! MEMORY FULL. A document that assumes every SysEx bulk-dump failure is 40, 41 or 42 gets this one wrong.

6.2 ERROR 40 — wrong identification

established Raised when the grammar fails at depth 2, 3 or 4, that is at message byte 3, 4 or 5. For a message addressed to the WSA1 (0x2D) this means byte 3 was not 0x04, or byte 4 was not 0x00/0x01, or byte 5 was not 0x11.

6.3 ERROR 41 — reception

established The catch-all. The conditions actually raised include: first byte of a message in a session is not 0xF0; second byte is not 0x50; a status byte other than 0xF7 arrived inside an open message; the UART reported a framing, parity or overrun error; a reply timed out; the message exceeded 254 bytes; an odd body length; the continuation flag was neither 0x00 nor 0x01; the checksum did not match.

6.4 ERROR 42 — transmission

established Exactly one reachable cause. Status 0x20 is raised in the loop that streams a dump out, when the inter-processor link reports that a block transfer is still outstanding about 1.02 s after the wait began. Statuses 0x18 and 0x1F also map to ERROR 42 but are written nowhere in any of the four ROM images.

6.5 Where errors are visible

established Status bytes are written whenever a complete message is parsed, from either MIDI ring, but the popup appears only from the SYSEX BULK DUMP screen. A foreign SysEx arriving on any other screen sets no visible error.

established While a bulk-dump session is open, unrelated SysEx traffic on the MIDI input can set the status and so colour the popup the session ends with.

6.6 Verbatim screen text

established The firmware’s own spelling is reproduced here as it appears:

ERROR 40! — “The Identification (ID) code of the System Exclusive data received by the WSA is different product.”

ERROR 41! — “An error has occurred during System Exclusive data reception. The data from the transmitting device may be incomplete. Please try again.”

ERROR 42! — “An error has occurred during System Exclusive transmission. The data has not been received correctly. Please try again.”

established A complete German translation of all three screens exists in `prom_b` but is unreachable: the painter hard-codes the English table, and the language selector byte is written once, to zero, during reset.

Appendix A

Evidence index

Every finding behind this manual, with the firmware symbols that establish it.

A.1 Technics SX-WSA1R ? Universal / General MIDI System Exclusive (receive and transmit), from the annotated TLCS-

topic	certainty	symbols
HEADLINE: only TWO universal SysEx messages are recognised ? GM System On and GM System Off	established	LinkTable_F4FF61 root sub-array at 0xF5115B (records 767-781), PtrTable_F4F888 entries 32/33, PtrTable_F4F800 entries 32/33, handlers 0xFB51E7 and 0xFB520C
GM System On ? exact bytes and exactly what it changes	established	handler 0xFB51E7 (PtrTable_F4F800[32] = PtrTable_F4F888[32] = 0x00FB51E7); poster 0xF41B18 -> prom_a sub_F8BC08 0xF8BC08 -> sub_F8BF07 0xF8BF07 -> T_Queue2E00_AppendRegs
GM System Off ? exact bytes, and it is a no-op when GM mode is already off	established	handler 0xFB520C (PtrTable_F4F800[33] = PtrTable_F4F888[33] = 0x00FB520C); .LFB523E is the early-out
Bit 2 of RAM 0x7F4D IS the GENERAL MIDI MODE flag (corroborated four independent ways)	established	Paint_GeneralMidiMode 0xF99D0C, Paint_GeneralMidiMode_Entry 0xF99844, ScreenButton_GeneralMidiMode_Entry 0xF9984C; prom_b sub_F45119 0xF4514F/0xF45156; 0xF64B88/0xF64B97;
Where the message is decoded: a byte-trie whose FIRST level matches the THIRD byte of the	established	sub_FB63D1 0xFB63D1, sub_FB6219 (record setter) 0xFB6219, sub_FB62D3 (record getter) 0xFB62D3, LinkTable_F4FF61 records 18/19 (0xF4FFCD) and 15/16/17 (0xF4FFBB), PtrTable
The device-ID byte must be 0x7F exactly ? there is no per-unit device number and no wildca	established	sub_FB63D1 level-1 compare at 0xFB6415; level-2 compare with wildcard at 0xFB64C5/0xFB64C9; root list 0xF5115B..0xF511AF

topic	certainty	symbols
NOT recognised: Master Volume, Identity Request/Reply, GM2 On, and every other universal m	established	MIDI_RX_SysExStart 0xFA584D (interrupt side), MIDI_Fg_SystemCommon 0xFA5B15 (foreground side), MIDI_SysEx_Ignore 0xFA586D, level-2 list at 0xF4FFCD, level-3 list at 0xF4F
GM On/Off are NOT subject to the per-command receive filter	established	sub_FB6B87 0xFB6B87, class table 0xF4FE82 (34 bytes, 0xF4FE82..0xF4FEA3; bytes[32]=0x00, bytes[33]=0x00), sub_FB6BF4 0xFB6BF4, sub_FB6CFC 0xFB6CFC, sub_FB6DBD 0xFB6DBD
The receive framing: which bytes must be present, and where the message is assembled	established	sub_FB20CE 0xFB20CE (drains ring 0x601646, dispatch through sub_FB2160 + PtrTable_F4F800) and sub_FB21CB 0xFB21CB (drains ring 0x601C6E, dispatch through sub_FB225D + Ptr
Both MIDI receive paths reach the same handlers	established	main loop 0xF82077 ('call 0xf408e8' -> T_F408E8 -> 0xFB20CE) and 0xF820D3 ('call 0xf40904' -> T_F40904 -> 0xFB21CB), gated by Ring601C6E_IsEmpty 0xF41EB0 at 0xF820CB; T_R
The unit also TRANSMITS GM System On / GM System Off over MIDI	established	template pair at 0xF4FEE6 / 0xF4FEEC (part of the 0xF4FEA8-0xF4FF60 SysEx template block); sender sub_FB4CAE 0xFB4CAE; transmit chain sub_FB6F24 0xFB6F24 -> sub_FB71BB 0x
The same two messages are written into Standard MIDI Files, selected by the GM-mode flag	established	GmSystemSysEx_F76E64 (0xF76E64 / 0xF76E6C), chosen at 0xF76F4A 'ld XIY,0x00F76E64' + 0xF76F4F 'bit 2,(0xf74d)'; Data_F75675 (0xF75675 / 0xF7567D), chosen at 0xF7575B + 0x
Trailing bytes before the F7 are probably tolerated	likely	sub_FB63D1 exit .LFB6B6C 0xFB6B6C; dispatch gate in sub_FB225D at 0xFB227A ('cp A,0x00' on slot 4) and 0xFB2291 ('cp A,0x22' on slot 0)
The manufacturer-ID byte is not re-checked after the front end	likely	sub_FB63D1 0xFB63D1 (the +2 skip at 0xFB63E1), root list 0xF5115B, front-end filters MIDI_RX_SysExStart 0xFA5850 and MIDI_Fg_SystemCommon 0xFA5B15
A global kill switch on SysEx reception: (0x89) == 0xFF	established	sub_FB20CE 0xFB20D1, sub_FB21CB 0xFB21CE; the byte (0x89)
Context for the manual: the rest of the SysEx engine is Matsushita ID 0x50	established	template block 0xF4FEA8-0xF4FF60 (readers at 0xFB28D9, 0xFB28F1, 0xFB279C, 0xFB27CD, 0xFB2812, 0xFB28E9, 0xFB2334, 0xFB23AD, 0xFB7034 and the 0x2D family 0xFB2429..0xFB27
Provenance note: the task's two named entry points are on the transmit/UI side, not the re	established	GmSystemSysEx_F76E64 (prom_b/wsa1_prom_b.s:167308), Paint_GeneralMidiMode_Entry (prom_a/wsa1_prom_a.s:42886), Paint_GeneralMidiMode 0xF99D0C

topic	certainty	symbols
-------	-----------	---------

A.2 SX-WSA1R MIDI transport on CPU 1 (prom_a/prom_b): how MIDI 1 and MIDI 2 are distinguished, what confines BULK

topic	certainty	symbols
There are two MIDI input pumps and they differ by ONE byte: the port tag	established	MidiIn_PumpPortA 0xFA6018, MidiIn_PumpPortB 0xFA60C2, MidiIn_FetchMessage_PortA 0xFA609A, MidiIn_FetchMessage_PortB 0xFA6123, staging record 0x1940-0x1943, T_MidiIn_PumpP
The port tag is bit 4 of a 5-bit channel index: 32 logical channels, 16 per port	established	MidiIn_RouteChannelMessage 0xFA61CE, tables 0x1800 (32 B) and 0x1820 (64 B), second pair 0x18A0/0x18C0, cursor/count cells 0x1974/0x1975/0x1977, current index 0x197E
MIDI 1 is CPU 1's own UART; MIDI 2 arrives pre-parsed over the inter-CPU link	established	MIDI_RX_Byte 0xFA5496, MIDI_RX_DeliverTwo 0xFA57AF, MIDI_RX_DeliverThree 0xFA5803, MIDI_DrainQueue 0xFA5942, MIDI_Fg_Deliver2 0xFA5A8B, MIDI_Fg_Deliver3 0xFA5ABE, MIDI_Po
That the far end of the link is the physical MIDI 2 jack is an inference, not a schematic	likely	prom_c_INTRX0_HANDLER 0xF991FF, INTTX0_HANDLER 0xF99265, Link_Ch2_ForwardBytes (per-byte sink of link channel 2) 0xF992C6, MIDI_Tx_PutByte, prom_c 0xFCC5C2-0xFCC5C4 (FF/F
SYSTEM EXCLUSIVE never travels in the port-A/port-B message rings at all	established	MIDI_RX_SysExStart 0xFA584D, MIDI_RX_SysExData 0xFA586E, MIDI_Fg_SystemCommon 0xFA5AEB, MIDI_Fg_SysExData 0xFA5B3D, MidiIn_SystemMessage 0xFA614B, MidiIn_SystemSubTable 0
Two SysEx identifiers are accepted and only two: 0x50 and 0x7E	established	MIDI_RX_SysExStart 0xFA584D (‘cp E,0x50’ FA5850, ‘cp E,0x7E’ FA5855), MIDI_Fg_SystemCommon 0xFA5B15/0xFA5B1A, SysEx assemblers 0xFB210E/0xFB2113 and 0xFB220B/0xFB2210, bu
State bytes: (0xA9) interrupt-side, (0x0964) foreground-side, same bit layout	established	(0xA9) SysEx state (interrupt), (0x0964) SysEx state (foreground), (0x9A)/(0x0960) running status, (0x9E)/(0x0963) flags, (0x216F) bit 0 = MIDI activity
The two SysEx assemblers are twins, one per port - so ORDINARY SysEx reception is not MIDI	established	sub_FB20CE (MIDI 1, ring 0x601646 via 0xF41E38), sub_FB21CB (MIDI 2, ring 0x601C6E via 0xF41EA4), sub_FB6165 (byte appender), sub_FB2160 (post-0xF7 processor), thunks T_F

topic	certainty	symbols
What confines BULK DUMP to MIDI 1: the dump's own send and receive loops name only the M	established	sub_FB7165 (MIDI 1 only) 0xFB7165, sub_FB71BB (both ports) 0xFB71BB, bulk send drivers sub_FB6F6B 0xFB6F6B / sub_FB6FB2 0xFB6FB2, bulk send dispatcher sub_FB23E2 0xFB23E2
Ring 0x601432 is MIDI 1 OUT, ring 0x60153C is MIDI 2 OUT, and (0x194B) is the runtime port	established	MIDI_TX_Ready 0xFA542F, Ring601432_Get 0xF41DF0, MidiOut_PutByteA 0xFA7D95, MidiOut_PostStagedMessage (the 0xFA7D26 site), (0x194B) port select, MIDI_SendBankAndProgram 0
Every buffer size, from the ring class itself	established	ring class 0xF84000-0xF842DE, instance bank 0xF842DF-0xF84C6B (15 groups of 11 veneers, 163 bytes each), free-count field at ring-2 (0x601430 for port A, 0x60153A for por
Input overrun: four distinct failure modes, three counters, one flush	established	MIDI_RX_ErrorReset 0xFA5418, MIDI_RX_QueueFull2 0xFA57D0, MIDI_RX_QueueFull3 0xFA5828, MIDI_RX_SecondDataByte 0xFA57DE, error counter (0x0931) 8-bit, overflow counter (0x
Output overrun: 240 polls, then the whole transmit queue is thrown away	established	MidiOut_PostStagedMessage (0xFA7CEB-0xFA7D91), retry counter (0xA2), SC0BUF = SFR 0x50
SysEx accumulation has its own 255-byte ceiling	established	sub_FB6165 0xFB6165, descriptor pointer (0x60FC80), count at +0x00, write cursor at +0x0A, abort sites 0xFB212B/0xFB2228/0xFB610A
The bulk-dump packet format, read out of the transmit engine	established	sub_FB6E7F 0xFB6E7F (header), sub_FB6F24 0xFB6F24 (raw append), sub_FB7040 0xFB7040 (nibbliser), sub_FB70AE 0xFB70AE / sub_FB70E6 0xFB70E6 (continuation flag), sub_FB7111
The bulk-dump address and length fields are 3 bytes of base-128, MSB first - proved by a	established	templates prom_b 0xF4FEF8, 0xF4FF04, 0xF4FF10, 0xF4FF1C, 0xF4FF28, 0xF4FF34, 0xF4FF40, 0xF4FF49, 0xF4FF55; setters sub_FB75BA 0xFB75BA and sub_FB75E4 0xFB75E4; senders su
The other fixed SysEx templates in ROM	established	template table prom_b 0xF4FEB4..0xF4FF60, readers sub_FB7025 0xFB7025 (0xF4FED2), 0xFB33A4 (0xF4FEE3), 0xFB2429/0xFB246E (bulk), sub_FB6E7F 0xFB6E7F
The received-SysEx command dispatch: a 34-entry table at prom_b 0xF4F800	likely	sub_FB2160 0xFB2160, sub_FB62D3 0xFB62D3 (getter, JumpTable_FB62F6), sub_FB6219 0xFB6219 (setter, JumpTable_FB6240), command table prom_b 0xF4F800, record pointer (0x60FC

topic	certainty	symbols
The MIDI settings that touch SysEx: an EXCLUSIVE filter, and no device number	likely	filter readers sub_F9ABEF 0xF9ABEF, sub_F9AC15, sub_F9AC3B, sub_F9AC61, sub_F9AC87, sub_F9ACAA, sub_F9ACD0, sub_F9ACF6 0xF9ACF6; editors sub_F9AD21..sub_F9AF19; DL_InputO
Two settings bits that do reach the MIDI code, and their consumers	established	MidiIn_SongPosition 0xFA61A4, MidiIn_SongSelect 0xFA61B8, MidiIn_ProgramChange 0xFA6D58, MidiIn_PitchBend 0xFA6DE3, MidiIn_ChannelPressure 0xFA6E6A, MidiIn_ControllerEnab

A.3 Technics SX-WSA1R firmware ? MIDI System Exclusive RECEIVE path (prom_a interrupt parser, prom_a foreground pa

topic	certainty	symbols
How F0 reaches the SysEx handler at all	established	MIDI_RX_Byte 0xFA5496; MIDI_RX_DataByte 0xFA5763; MIDI_StatusDispatch_Table 0xFA578E; MIDI_RX_SystemCommon 0xFA5830; MIDI_RX_SysExStart 0xFA584D
The identifier test ? exactly two bytes, both fixed immediates	established	MIDI_RX_SysExStart 0xFA584D; MIDI_SysEx_Accept 0xFA585A
What makes it take the Ignore path, and what Ignore actually does	established	MIDI_SysEx_Ignore 0xFA586D; MIDI_RX_SysExData 0xFA586E
On Accept, F0 and the identifier are re-emitted into a ring, not just flagged	established	MIDI_SysEx_Accept 0xFA585A; T_Ring601646_Put 0xF41E3C; Ring601646_Put 0xF849EE
Body-byte gate: forwarded only while bit 1 set AND bit 5 clear	established	MIDI_RX_SysExData 0xFA586E; MIDI_SysEx_DataDone 0xFA5883
Termination and the abort rule ? how a message ends (0xA9) ? the complete SysEx state byte, and a CORRECTION to the existing annotation	established	MIDI_RX_Byte .LFA54C0 0xFA54C0 / .LFA54E5 0xFA54E5 / .LFA54EE 0xFA54EE
System Real Time is suppressed while a SysEx is in flight	established	MIDI_RX_ErrorReset 0xFA5426; sub_FB6098 0xFB6098 (set at 0xFB6151); MIDI_RX_SysExStart 0xFA584D
	established	MIDI_RT_Received 0xFA5504; sub_FB99AB 0xFB99AB

topic	certainty	symbols
MIDI_SysExHeader (F0 50) is a TRANSMIT template ? ZERO bytes are ever matched against it	established	MIDI_SysExHeader 0xFA5CB8; MIDI_Fg_SystemCommon 0xFA5AEB; T_Ring601C6E_PutBlock 0xF41EAC
A second, independent copy of the same state machine runs in the foreground	established	MIDI_DrainQueue 0xFA5942; MIDI_Fg_SystemCommon 0xFA5AEB; MIDI_Fg_SysExData 0xFA5B3D; MIDI_Fg_DataByte 0xFA59FA
Where received data is buffered, and exactly what bounds it	established	sub_FB20CE 0xFB20CE; sub_FB21CB 0xFB21CB; sub_FB6165 0xFB6165; state byte (0x60FD43); buffer pointer (0x60FC80)
The buffer objects themselves	likely	sub_FB7F42 0xFB7F42; sub_FB7FEB 0xFB7FEB; sub_FB8028 0xFB8028; sub_FB80F3 0xFB80F3 (reset the active buffer)
The grammar walker SKIPS F0 and the identifier without re-comparing them	established	sub_FB63D1 0xFB63D1; sub_FB61B1 0xFB61B1 (read one byte, advance the cursor)
The accepted third byte ? the root table, 14 values	established	root table 0xF5115B (prom_b Link-Table_F4FF61 record 767); sub_FB63D1 0xFB63D1; probe wsal/notes/sysex-probes/sysex_grammar_dump.py
The complete short-form accepted sequences, with their command numbers	established	root 0xF5115B and nodes 0xF4FF67..0xF511B4; literal templates 0xF4FEB4-0xF4FF60; probe wsal/notes/sysex-probes/sysex_grammar_dump.py -paths
The byte after 0x04 is a two-valued unit/device field; 04 nn 11 is a model triple	established	nodes 0xF4FF85/0xF4FF73/0xF4FF67 (for 0x21) and 0xF4FFAF/0xF4FF9D/0xF4FF91 (for 0x22); sub_FB2323 0xFB2323; sub_FB7F1F 0xFB7F1F
Validation: the parse record, its error codes, and the dispatch bound	established	sub_FB62D3 0xFB62D3 (field getter); sub_FB6219 0xFB6219 (field setter); sub_FB2160 0xFB2160; sub_FB225D 0xFB225D; template 0xF511E9
A third, stricter collector accepts ONLY 0x50	established	sub_FB6098 0xFB6098
Two annotations in the same file disagree about what 0x50 is	unverified	header at 0xFA5919-0xFA5938 (MIDI_RX_SysExStart); header at ~0xFA5AEB (MIDI_Fg_SystemCommon); notes/gen_prom_a_fa5aeb_module.py line ~515

A.4 WSA1R SysEx error reporting ? where ERROR 40 / 41 / 42 are raised (prom_a + prom_b)

topic	certainty	symbols
The whole chain: status byte → message id → screen	established	sub_FB7DFE 0xFB7DFE, sub_FB7E29 0xFB7E29, sub_FB7E38 0xFB7E38, sub_FB7E43 0xFB7E43, AsciiRun_F511C7 0xF511C7 (prom_b), PtrTable_F99121 0xF99121, sub_F99098 0xF99098, sub_
ERROR 40 ? exact condition	established	sub_FB63D1 0xFB63D1; raise sites 0xFB64BF (0x08), 0xFB6582 (0x09), 0xFB6645 (0x0A); shared tail 0xFB6ADA; root table 0xF5115B (prom_b)
ERROR 42 ? exact condition, and it is a SEND-side error only	established	sub_FB6FB2 0xFB6FB2, wait at 0xFB6FD1 'call 0xf4123c', raise at 0xFB6FFB and 0xFB700A; Link_WaitBlockDone 0xF8E66D; thunk T_Link_WaitBlockDone 0xF4123C; sub_FB6F6B 0xFB6F
ERROR 41 ? the catch-all, and the full list of what lands on it	established	0xFB60E9(01) 0xFB60F7(02) 0xFB6142(03) 0xFB776D(04) 0xFB77A1(05) 0xFB611B(06) 0xFB640C(07) 0xFB6708(0B) 0xFB67CB(0C) 0xFB688E(0D) 0xFB6951/0xFB6CE5(0E) 0xFB6A14(0F) 0xFB6
The checksum, and the three-byte tail every accepted message must have	established	sub_FB6CFC 0xFB6CFC, sub_FB6DBD 0xFB6DBD; product-id gate 0xFB6DE8-0xFB6DFA; flag test 0xFB6E09-0xFB6E0F; sum loop 0xFB6E3A-0xFB6E47; checksum compare 0xFB6E4D-0xFB6E62
Body encoding: two 7-bit bytes per data byte, high nibble first	established	receive join 0xFB72E2 'sll h,0x04' + 0xFB72FC 'and A,0x0f' + 0xFB7301 'xor L,H'; transmit split sub_FB7040 0xFB7040, 0xFB706F / 0xFB7074
Status 0x16 is NOT one of the 4x errors ? it shows ERROR 21! "Memory full"	established	raise sites 0xFB734C and 0xFB7401; message 0x0F → 0xF2DD4F
TRAP: the .s under-declares STATUS_MAP as 22 bytes; it is 34	established	AsciiRun_F511C7 0xF511C7, header at prom_b/wsa1_prom_b.s line ~101343
The three errors are reported ONLY by the SYSEX BULK DUMP screen	likely	sub_FB7DFE callers 0xFB20BF and 0xFB286F; engine entry sub_FB2049 0xFB2049; JumpTable_FB2081 (6 modes, 'cps bc,0x05 / jr ugt' at 0xFB2070); screen painter Paint_SysexBulk
The messages the machine accepts at all, and the WSA1's own product id	established	root 0xF5115B; templates 0xF4FEB4-0xF4FF60; handler tables 0xF4F800 / 0xF4F888; bounds 0xFB2194 / 0xFB2291
Before the trie: the session framer's own three checks	established	sub_FB6072 0xFB6072, sub_FB6098 0xFB6098; sub_FB20CE 0xFB20CE (ring 0xF41E38), sub_FB21CB 0xFB21CB (ring 0xF41EA4, Ring601C6E_Get); byte appender sub_FB6165 0xFB6165
Four statuses exist in the map but are never raised	established	STATUS_MAP 0xF511C7 indices 0x15, 0x17, 0x18, 0x1F

topic	certainty	symbols
A complete GERMAN copy of all three error screens exists but is unreachable	established	PtrTable_F99121 0xF99121 (+ base table 0xF993B9); PtrTable_F98DE5 0xF98DE5 (+ base table 0xF98FB1); selector (0x7FC1) written at 0xF82628
What the screens actually say, verbatim	established	DL_Error40TheIdentificationIdCodeOfThe 0xF2E46A, DL_Error41AnErrorHasOccuredDuringSystem 0xF2E4E4, DL_Error42AnErrorHasOccuredDuringSystem 0xF2E587, DL_Completed 0xF2E615

A.5 Technics SX-WSA1R firmware ? the SysEx BULK DUMP transmit path (prom_a UI screen 0x79 → the SysEx engine at 0x

topic	certainty	symbols
Entry point: screen 0x79's button handler is what starts a dump	established	Paint_SysexBulkDump 0xF99A04, ScreenButton_SysexBulkDump_Entry 0xF99835, DisplayListPtrs_F99870, 0xF99A8F, 0xF99AE3, 0xF408E4 (T_F408E4), sub_FB2049 0xFB2049, JumpTable_F
Five menu rows, four data categories	established	0xF99AE3, sub_FB22CD 0xFB22CD, sub_FB22E6 0xFB22E6, sub_FB22E7 0xFB22E7, sub_FB22F6 0xFB22F6, sub_FB2305 0xFB2305, sub_FB2314 0xFB2314, sub_FB23E2 0xFB23E2, sub_FB5FF5 0x
Data message frame layout	established	templates prom_b 0xF4FEF8, 0xF4FF04, 0xF4FF10, 0xF4FF1C, 0xF4FF28, 0xF4FF34, 0xF4FF40, 0xF4FF49, 0xF4FF55; sub_FB6E7F 0xFB6E7F (append+autoflush), sub_FB6F24 0xFB6F24 (ap
The address and size fields ? proved, not assumed	established	sub_FB75BA, sub_FB75E4, sub_FB751A, sub_FB7649, sub_FB766F, sub_FB7692, sub_FB76B5, sub_FB76F2, sub_FB7722, sub_FB6EE9 0xFB6EE9
Data bytes are sent as nibble pairs, high nibble first	established	sub_FB7040 0xFB7040
Checksum: (0 - sum) & 0x7F over everything after the leading F0	established	sub_FB7111 0xFB7111, literal prom_b 0xF4FE68
Block size: 0xFC message bytes, so 120 or 125 source bytes per frame	established	sub_FB7040 0xFB70A3 'cp WA,0x00FC', sub_FB7165 0xFB71A0 'cp HL,0x0020', Ring601432_PutBlock 0xF8481C, Ring_Put_0100 0xF841D5
Continuation frames re-open with F0 50 7E	established	sub_FB7025 0xFB7025, template prom_b 0xF4FED2 = F0 50 7E, sub_FB70AE 0xFB70AE, sub_FB70E6 0xFB70E6, sub_FB8081 0xFB8081

topic	certainty	symbols
Continuation flag semantics, and the two frame loops	established	sub_FB6F6B 0xFB6F6B, sub_FB6FB2 0xFB6FB2, sub_FB70AE 0xFB70AE, sub_FB70E6 0xFB70E6, Remote_E80000_Read32Blocks 0xFB24EC, sub_FB26E7 0xFB26E7
Handshake: a two-message enquiry before any data, retried three times each	established	sub_FB2323 0xFB2323, templates 0xF4FED5 and 0xF4FEDC, sub_FB6072 0xFB6072, sub_FB62D3 0xFB62D3, (0x60FD40) bit 7
Per-frame acknowledgement, retransmission and the two timeouts	established	sub_FB6026 0xFB6026, sub_FB6072 0xFB6072, sub_FB7785 0xFB7785, sub_FB77C7 0xFB77C7, Ring601432_SpinUntilEmpty 0xFB6084, sub_FB7165 0xFB7165
End-of-category, end-of-dump and abort messages	established	sub_FB277E 0xFB277E, sub_FB27AD 0xFB27AD, templates 0xF4FEBC, 0xF4FEC3, 0xF4FEC8
The same protocol in reverse ? what the WSA1R answers when it is the receiver	established	sub_FB28BE 0xFB28BE, 0xFB28FF, 0xFB291D, dispatch table prom_b 0xF4F916 (34 entries, bound 'cp H,0x22' at 0xFB2896)
The complete command byte vocabulary the firmware will parse	established	trie root prom_b 0xF5115B, sub_FB63D1 0xFB63D1, sub_FB61B1 0xFB61B1, wsal/notes/sysex-probes/sysex_grammar_dump.py
The receiver only accepts messages that begin F0 50	established	sub_FB6098 0xFB6098, sub_FB6165 0xFB6165, MIDI_RX_SysExStart 0xFA584D
How a frame reaches the wire, and which port	established	sub_FB7165 0xFB7165, sub_FB71BB 0xFB71BB, T_Ring601432_PutBlock 0xF41DF8, T_MIDI_PostSendWork 0xF40724, Ring601432_Get 0xF847F7, MIDI_TX_Ready 0xFA542F
Per-category transfer breakdown as it appears on the wire	established	sub_FB23EF/sub_FB23F9/sub_FB243E, Remote_E80000_Read32Blocks 0xFB24EC, sub_FB2675/sub_FB267F/sub_FB26E7, sub_FB2587/sub_FB25A2/sub_FB25E7/sub_FB262C, descriptor writers s
The buffers and the parse record, for anyone re-deriving this	established	sub_FB7F42 0xFB7F42 (the allocator), sub_FB7FEB 0xFB7FEB (buffer init, descriptor template prom_b 0xF511E9), sub_FB6219 0xFB6219 (field setter), sub_FB62D3 0xFB62D3 (field setter)
The progress display and its status codes	established	Paint_Sending 0xF99B82, Paint_SysPartMidiSoundCombination 0xF99C5C, sub_FB7B0B 0xFB7B0B, sub_FB7B79 0xFB7B79, pointer array prom_b 0xF0D942
The 2B / 2C messages are single-parameter traffic, not bulk dump	likely	template prom_b 0xF4FEF2, transmit sites 0xFB47E9, 0xFB48E7, 0xFB4A59, 0xFB4DAF, 0xFB4E65, 0xFB4F26, 0xFB4FDC, 0xFB53BF, 0xFB5473; dispatch table prom_b 0xF4F800 entries

topic	certainty	symbols
Remote-triggered dumps use the same engine	likely	0xFB5122-0xFB514A, sub_FB516A 0xFB516A, sub_FB5154 0xFB5154, dispatch tables prom_b 0xF4F800 / 0xF4F888

Appendix B

Open questions

What this edition does not establish. These are recorded so a later pass knows where the evidence stopped.

- What bit 7 of (0x60F020) means. Both GM handlers set it before posting their event, and nothing converted in this tree reads it.
- What consumes the {E=0x91, D=0x03, A=<(0x7F4D)>, W=0x04} record on the 0x2E00 pending list. Until that is traced, 'GM System On replaces the current settings with the GM settings' is the panel dialog's claim, not something the disassembly establishes for the SysEx path.
- What sets byte +1 of sub_FB4CAE's argument record to 0x11 or 0x10 ? i.e. what actually makes the unit TRANSMIT GM System On/Off over MIDI (as opposed to writing it into an SMF). The template selection is proven; the trigger is not.
- What (0x89) is and who sets it to 0xFF. While it is 0xFF, both SysEx drainers exit immediately and no SysEx at all is processed.
- Whether trailing bytes between the sub-ID#2 and the F7 are genuinely tolerated. Needs a hardware test: send F0 7E 7F 09 01 00 F7 and see whether GM mode turns on. Hardware is currently unreachable ? park it in kn7000_mame/notes/HARDWARE-QUESTIONS-PENDING-FELIPE.md rather than asserting it.
- Whether F0 50 7F 09 01 F7 really acts as GM System On (the ID byte is never re-checked after the front-end filter). Same hardware test applies.
- Which company holds MIDI manufacturer ID 0x50. The tree refuses to name it without the MMA assignment list; the published manual should do the same.
- What the Matsushita commands 1-33 do individually. Only the two GM slots were traced here; PtrTable_F4F800 slot 9 (0xFB33FE) and slots 23-31 have distinct handlers that were not read.
- Data_F73844 and Data_F75675 (prom_b) are still classified as unidentified data blocks even though they are byte-identical SMF GM-event pairs with located readers. Their headers should be corrected in the disassembly; that is a source edit this pass did not make.
- No reproducibility artefact was committed for this pass. The enumeration above was done by reading the .s files plus one throwaway Python byte-scan of original_ROMs/*. If any of these byte sequences are published, a small committed probe (walk LinkTable_F4FF61 from record 767 and print every accepted message) belongs beside the disassembly, per the project's permanent-storage rule.
- The manual's 'BULK DUMP only on MIDI 1' is explained by call sites, not by a setting: sub_FB7165 (send) names only ring 0x601432 and sub_FB6098 (receive) names only ring 0x601646. But ORDINARY SysEx reception exists on both ports (sub_FB21CB drains the MIDI 2 SysEx ring 0x601C6E every main-loop pass). If the manual also says non-bulk SysEx is MIDI-1-only, that is NOT what the firmware does and the

- Which company holds manufacturer ID 0x50 is not settled in this tree, and the tree contradicts itself: the header at prom_a 0xFA5AEB says 'Matsushita', notes/FINDINGS-midi-port.md explicitly refuses to name one. Needs the MMA assignment list.
- That CPU 2's serial channel 0 is the physical MIDI 2 jack is an inference from the link-channel wiring (link selector 2 = CPU1->CPU2 MIDI bytes, link index 6 = CPU2->CPU1 MIDI bytes). The schematic pin table in notes/DRIVER-INSIGHT-wsa1-2026-09-02.md names only MIDI1OUT/MIDI1IN/MIDI1SNS, and that table has already needed one off-by-one correction. A MIDI2 pin label from the service manual would cl
- sub_FB63AF, which builds the 16-byte record that sub_FB2160 dispatches on, was not traced, so the mapping from wire bytes to record fields 0 and 4 is unknown. Until that is done, 'command number = record[0], 0x00-0x21' cannot be turned into 'the byte at wire offset N is the command'. The 34-entry table at prom_b 0xF4F800 is the next thing to enumerate for the manual.
- Frame-length arithmetic is one byte from a contradiction and should be measured, not reasoned about: the transmitter stops adding nibble pairs once the frame reaches 0xFC = 252 bytes and then appends flag + checksum + F7, while the receiver aborts a message whose accumulated count reaches 0xFF = 255. Whether the largest frame the machine emits is 255 or 256 bytes - and therefore whether it can ove
- The EXCLUSIVE field is edited with bit mask 0x0F but the transmit gates test only bit 3. The reading that ON writes all four bits (0x0F) and OFF writes 0x00 is consistent with both, but the bit-field setter at 0xFB... reached by 'calr 0xf22b' from 0xF9AF37 was not read, so it is inference. Do not publish 'EXCLUSIVE = (0x7F38) bit 3' as the storage format without reading that setter.
- sub_FB7025 conditionally prepends the 3-byte fragment F0 50 7E when (0x60FCE4)+0x0F == 1, inside the same per-packet loop that already has a 12-byte header in the buffer. Whether that is the continuation-packet header (and therefore whether packets 2..n look different from packet 1 on the wire) is not established.
- Reproducibility: the four-ROM address census quoted in the settings finding was run ad hoc as 'python3' over original_ROMs/*, scanning the 12 TLCS-900 direct-memory prefix spellings (0xC1/D1/E1/F1 with a 16-bit address field, 0xC2/D2/E2/F2 with a 24-bit one) for 0x7F32/33/35/36/38/39/3A/3B. notes/prom_a_xref.py and notes/prom_c_xrefs.py already do this properly for their own images; the cross-imag
- Who reads (0xA9) bits 2 and 4? They are written (0x04 after a clean F7, 0x10 after an aborted SysEx) but a census of every (0xA9) site in prom_a finds no test of either. If nothing reads them they are diagnostics only ? but a reader could live in prom_c/prom_d, which were not searched.
- Do the command HANDLERS re-check the manufacturer byte? The grammar walker provably skips message bytes 0 and 1, so at the grammar level 'F0 7E 21 04 00 11 F7' is accepted identically to 'F0 50 21 ...'. Many handlers do re-read the raw buffer descriptor (0x60FC80), so a late check is possible; none was located. This must be settled before a manual states that either identifier works.
- The tables contain command ids up to 0x27, but both dispatchers reject anything >= 0x22. Commands 0x22-0x27 therefore parse successfully and are then dropped by sub_FB2160/sub_FB225D. Is there a third dispatcher with a larger bound, or are those table entries dead? Only one site each references 0xF4F800 and 0xF4F888.
- What do commands 0x01-0x1F actually do? On the interrupt-side ring, 0x01-0x08 and 0x0A-0x16 all land on the single shared handler 0xFB2820, which gates on the panel-mode cell (0x207A)==0x79 and on command >= 7; on the foreground ring the same commands land on a no-op. Distinct handlers exist only for 0x00, 0x09, 0x17, 0x18, 0x19, 0x1A, 0x1B-0x1F, 0x20, 0x21.

- The 0x2B / 0x2C / 0x2D subtrees (338 reachable nodes) are the bulk of the protocol ? parameter addresses after '04 <00|01> 11' ? and were not enumerated. The committed probe dumps them with -paths; they need a pass of their own before any parameter table goes into a manual.
- What is the unit byte after 0x04 for ? a device number selectable in the UI, or a MAIN/SUB distinction between two linked units? The grammar accepts 0x00 and 0x01 unconditionally and the value is only remembered (0x60FC91), never compared. Felipe's hardware could settle this by watching what the instrument sends for its own unit.
- The 266-byte payload capacity per buffer is arithmetic on the 0x118 descriptor stride and the abutment at 0x60FC80, not a constant in the ROM. Worth confirming against a MAME memory map before it is printed as a limit; the byte-count bound of 255/256 bytes per message IS a ROM constant and is the number a manual should quote.
- (0x0964) = 4, set by the foreground parser after a clean F7, has no located reader ? the same open question the existing block header at 0xFA5942 already flags.
- The wording of ERROR 42 ("The data has not been received correctly") implies a handshake with the receiving MIDI device, but the only raise is a CPU1↔CPU2 link-block timeout (Link_WaitBlockDone). The tree's own header for Link_WaitBlockDone says of this very call site: "0xFB6FD1 ... what it waits on is NOT established". So whether the link block being waited on carries the outgoing MIDI stream to CP
- Semantically odd but repeatedly confirmed: an unknown PRODUCT ID at message byte [2] gives status 0x07 → ERROR 41, while a wrong byte at [3], [4] or [5] gives ERROR 40. If a real-hardware check is ever possible, sending F0 50 99 04 00 11 ... (bogus product) vs F0 50 2D 99 00 11 ... (good product, bogus next byte) would distinguish 41 from 40 on the panel and settle it. Felipe's hardware is currently u
- Trie depth 3 accepts 0x00 or 0x01 for every product id and both lead to the identical subtree. Reading this as a device/unit number is an INFERENCE (likely); the ROM only says the two values are accepted and equivalent.
- The tail flag byte before the checksum must be 0x00 or 0x01 (status 0x12 otherwise) and is stored in field 15 of the parse record. What it MEANS is not established ? nothing converted reads field 15 back.
- Statuses 0x19-0x1E each guard one bulk-dump section (their guard is 'field 3 of *(0x60FCDC) == 0', i.e. the previous section completed). Which section is which ? SOUND, COMBINATION, SYSTEM/PART & MIDI, TOTAL KEYBOARD, per the menu text in Paint_SysexBulkDump ? is NOT established and should not be guessed in the manual.
- PtrTable_F98DE5 (the German screen set) has no located reader; the tree's own header says so. Whether some other firmware variant of the WSA1R selects it, or it is dead weight from a shared source tree, is unknown.
- Reproducibility: the new probe `wsa1/notes/sysex-probes/sysex_error_codes.py` + its README section are committed as `kn5000-roms-disasm d4f11089` (with the required LLVM version line). It re-derives every number quoted above from `original_ROMs/`, asserts both load bases and prints OK. The pre-existing `wsa1/notes/sysex-probes/sysex_grammar_dump.py` (0ac8f694) covers the trie. Nothing from this session li
- Which incoming SysEx message actually selects a category for a remote-triggered dump. The five arms at 0xFB5122-0xFB514A are dispatch-table entries 0x1B..0x1F, but the receive trie at `prom_b 0xF5115B` produces no form with those classes. Either a second grammar root exists or those classes are raised internally; neither was found. UNVERIFIED ? do not publish a "dump request" message format.
- The class-6 arm of `sub_FB6026` (0xFB6041 'cp h,0x06' -> status 0x1F). No byte sequence in the trie yields class 6, so the wire form that triggers it is UNVERIFIED.

- Press/release semantics. PanelButton_Route (0xF861AC) passes the button index in W bits 0-4 and a flag in bit 7; the existing header reads bit 7 clear as "released", but that word is not cited to anything. On screen 0x79 the four row-select handlers act with bit 7 SET while the dump-start arm of 0xF99A8F acts with bit 7 CLEAR, and on the GENERAL MIDI screen (0xF99DDD, 0xF99DF4, 0xF99E2C) every han
- What the constant bytes 0x04 and 0x11 mean. They are in every 21/22/2B/2C/2D message and the trie matches them literally. A product-family byte and a model id is the obvious reading but nothing in the ROM says so. Likewise nothing says whether a user can ever make the machine transmit device number 0x01 rather than 0x00; every transmit template holds 0x00 and only the receiver accepts either.
- The address space. 0x080000 (SOUND), 0x100000 (SYSTEM,PART & MIDI), 0x140000 (COMBINATION), 0x180000/0x188400 (SEQUENCER) are logical addresses that do not equal the internal source addresses (RAM 0x7600, remote flash 0xE80000/0xEC0000). Whether they map onto anything a user or a service manual would recognise is UNVERIFIED.
- (0xC4). Its value 1 enables the SEQUENCER dump and 2 hides the SEQUENCER row entirely; a model or option variant is the obvious reading, but it is not established. This matters for the manual: on a machine where (0xC4) != 1 the SEQUENCER category is silently refused.
- The second output ring 0x60153C, woken through T_MIDI_PostSendWork_PortB (0xF40730 -> 0xFA5C12). It looks like a second MIDI OUT, and sub_FB71BB sends to both rings, but no bulk-dump frame goes through it and the port's identity is UNVERIFIED.
- The one-byte mailbox (0x9B), set to 0x00 by both sub_FB7165 and sub_FB71BB and to 0xF3 at 0xFE1507. notes/FINDINGS-midi-port.md records it as selecting an arm of MIDI_PostSendWork but does not decode it.
- The orphan at 0xFB248A and its template prom_b 0xF4FF10 (F0 50 2D 04 00 11 20 00 00 00 00 10). Nothing names 0xFB248A, and its template says 16 bytes while the routine passes count 0x0000 to the link read. It should NOT go into the manual as a real message; the probe asserts the anomaly so it will fire if either half changes.
- Frame timing on the wire. A 256-byte frame exactly fills the 0x100-byte MIDI-out ring and Ring_Put_0100 drops rather than blocks, so whether a real transfer ever loses a byte depends on interrupt latency, which cannot be settled from the disassembly. Felipe's hardware is unreachable (notes: kn5000-hardware-inaccessible), so this is a hardware question to park.